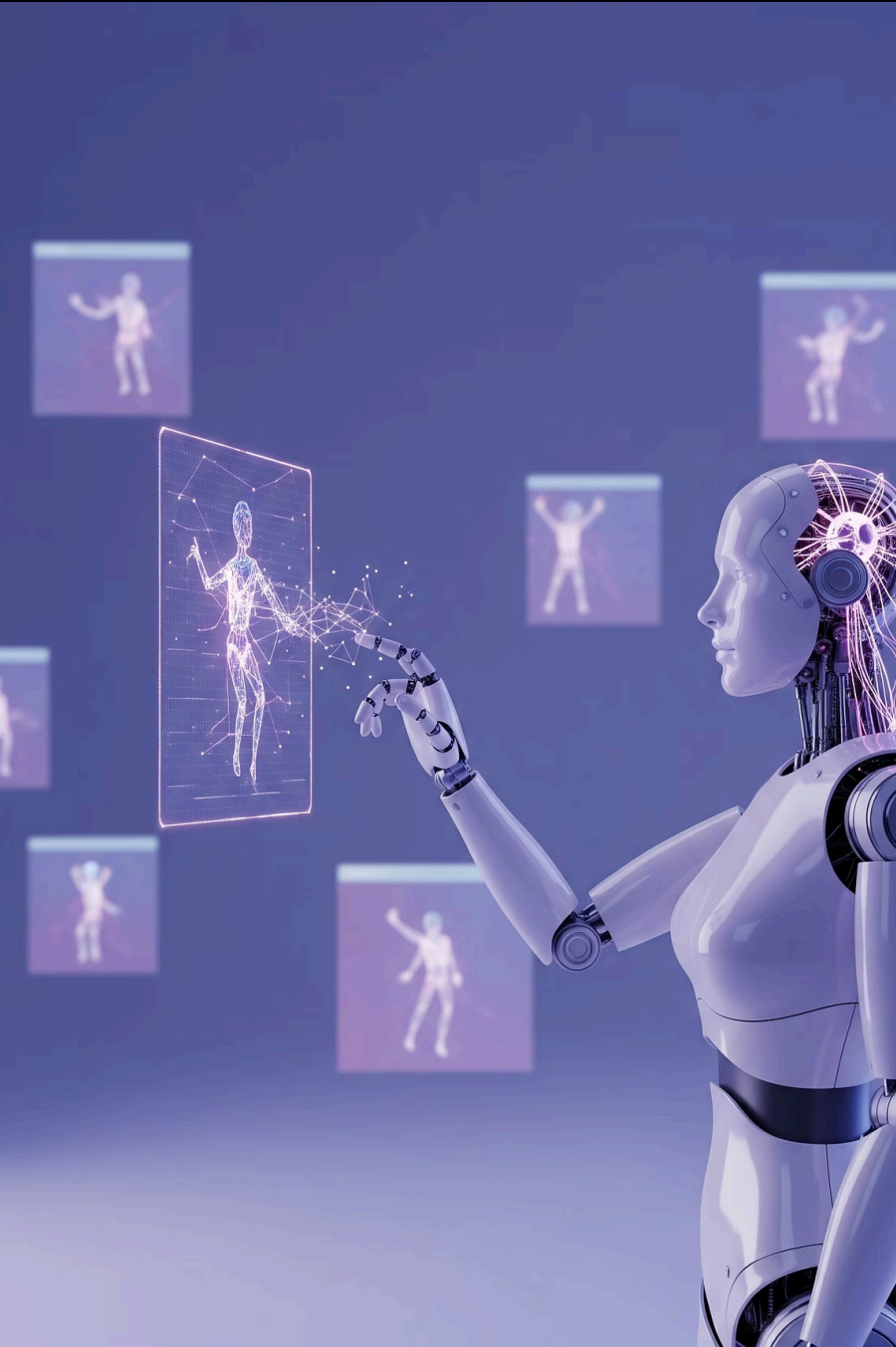


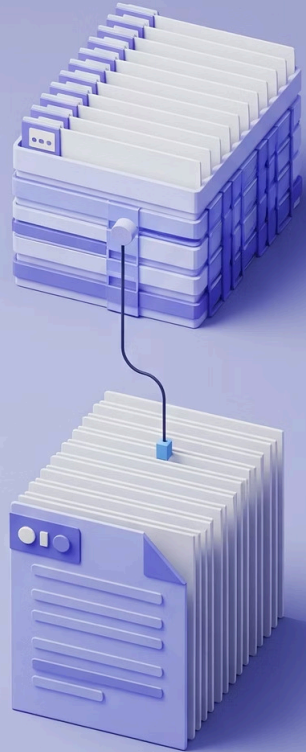


Basics of Reinforcement Learning (RL) Workloads

From Sim to Serve: Agents, Environments, and Infra

A practical guide for ML and infra engineers building RL training and serving pipelines

Machine Learning



VS

Reinforcement Learning



What Makes RL Different?

Data Generation

Data is **generated**, not **given** through agent-environment interaction loop

Reward-Based Learning

Feedback comes via **rewards**, not explicit labels

Shifting Distribution

Non-stationary data distribution as policy continuously evolves

Infrastructure Demands

Requires fast simulations, scalable rollout systems, and stable training mechanisms

Core RL Loop

1. Observe State
Agent receives state s_t from environment
4. Policy Update
Update policy/value networks to maximize expected return



2. Choose Action
Select action $a_t \sim \pi_{\theta}(\cdot|s_t)$ based on current policy
3. Environment Response
Environment returns next state s_{t+1} and reward r_t

i This fundamental loop differentiates RL from supervised learning - the agent's actions affect the data it sees next, creating a continuous feedback loop.

Common Algorithms (At a Glance)

Policy Gradient / PPO

On-policy approach with clipping for stability

Best for: General problems requiring stable convergence

Actor-Critic / A2C/A3C

Combines **parallel actors** with a value baseline

Best for: Distributed training across many CPUs

DQN / Rainbow

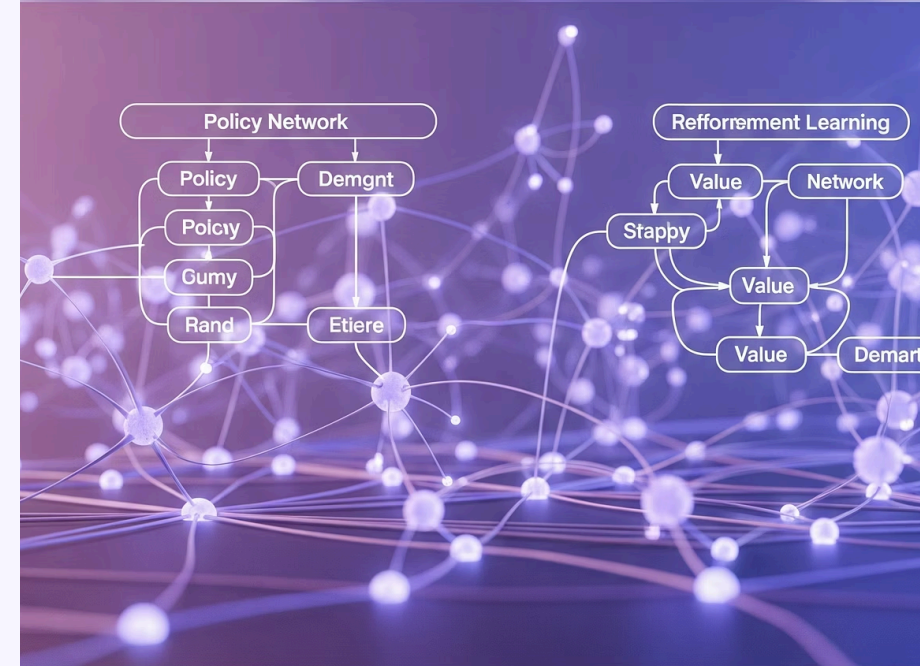
Value-based approach for **discrete action spaces**

Best for: Environments with clear state representations and limited action choices

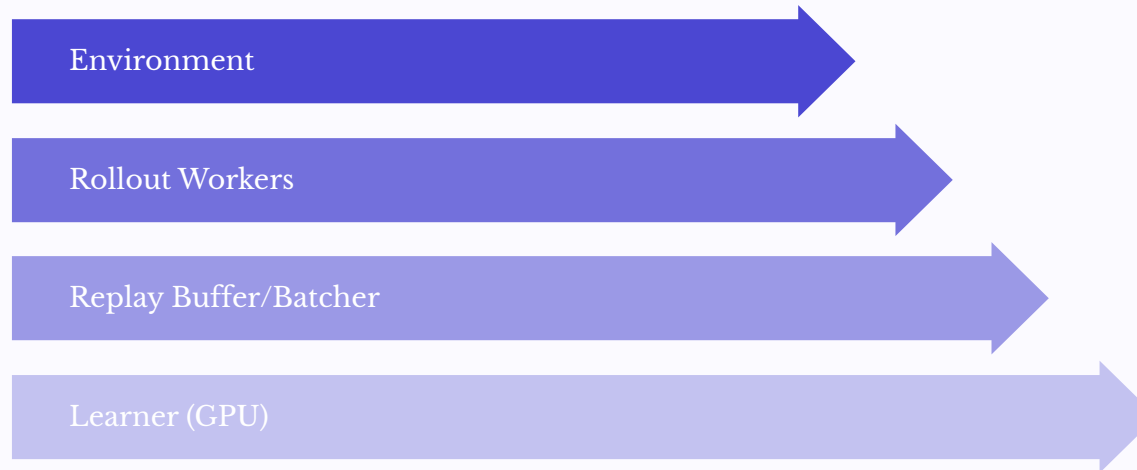
SAC/TD3

Off-policy algorithms for **continuous control**

Best for: Robotics and physical control tasks requiring precise movements



RL Workload Components



Environment & Rollout

- **Environment:** Gym/Isaac/Unity simulators
- **Rollout Workers:** Generate trajectories from agent-environment interaction

Learning & Evaluation

- **Replay Buffer/Batcher:** Store experiences for off-policy or on-policy learning
- **Learner:** GPU-accelerated neural network training
- **Evaluator:** Periodic checks of policy performance

Throughput & Scaling



Horizontal Scaling

Deploy many simulators in parallel to generate more experiences



Actor-Learner Separation

Actors (CPU) generate experiences while learners (GPU) update policy



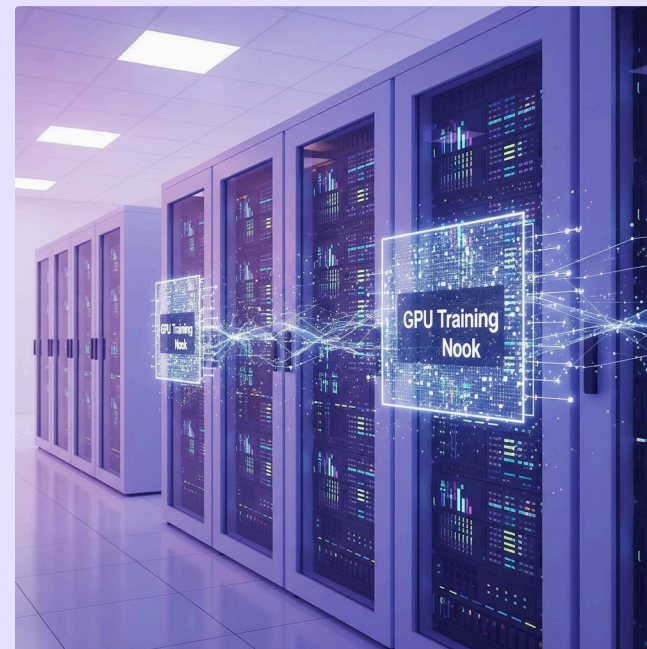
Asynchronous Queuing

Decouple simulation and training rates with efficient queues



Vectorized Environments

Process multiple environment instances simultaneously for increased throughput



Modern RL pipelines require infrastructure that balances [simulation throughput](#) with efficient learning

I/O & Serialization

Challenges

- Trajectories are **small but numerous** → high messaging overhead
- Image observations can create bandwidth bottlenecks
- Environment-agent communication can become a throughput limiter
- Reproducibility requires careful RNG management

Solutions



Optimize IPC

Use efficient inter-process communication for local systems



Compress Observations

Reduce bandwidth with image/tensor compression when needed



Shared Memory

Implement zero-copy transfers where possible for maximum throughput



Version Everything

Track RNG seeds and versions for reproducible experiments



Training Stability



Normalization & Clipping

Normalize rewards/observations and clip gradients to prevent divergence



Curriculum Learning

Gradually increase task difficulty as agent improves



Target Networks

Slow-moving target networks provide stability in value-based methods



Early Stopping

Monitor evaluation metrics and stop training when performance plateaus

⚠️ RL is notoriously unstable! Small hyperparameter changes can lead to dramatically different outcomes. Always monitor training closely.

Infra & Tooling

Training Frameworks

- **Ray RLlib:** Distributed RL with built-in algorithms
- **CleanRL:** Simple, clean implementations for research
- Custom frameworks for specialized workloads

Experiment Tracking

- **Weights & Biases:** Track metrics, hyperparameters, videos
- **MLflow:** Log artifacts and manage experiment lifecycle
- Custom monitoring for environment-specific metrics

Infrastructure

- **Docker containers:** Package environments with dependencies
- **Kubernetes/Slurm:** Orchestrate large-scale training runs
- GPU acceleration for learners, CPU farms for rollouts



Evaluation & Metrics

Return

Episodic Reward

Primary metric: cumulative reward per episode

Steps

Sample Efficiency

Reward per environment step (higher is better)

Time

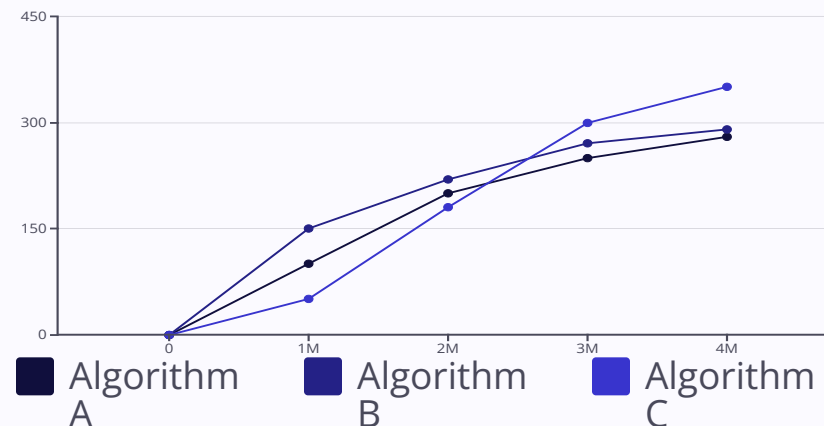
Wall-Clock Efficiency

Reward per hour of training or per dollar spent

%

Generalization

Performance on unseen environment variations



Comparing algorithms across training steps

Serving RL Policies

Model Export

Convert trained policies to optimized formats: [TorchScript](#), [ONNX](#), or [TensorRT](#)

Real-Time Control

Implement high-frequency control loops (10-1000 Hz) with strict latency bounds

Safety Measures

Add action clipping, boundary checks, and fallback controllers for robustness

Monitoring & Updates

Track distribution shifts and enable careful online policy updates when needed



⊗ Real-world RL deployment requires additional safety considerations beyond what's needed in simulation!

Key Takeaways



Fast Simulation + Stable Learning

RL workloads depend on efficient environment simulation coupled with stable learning algorithms to succeed



Logging & Reproducibility

Comprehensive metrics, careful versioning, and reproducible experiments are mission-critical for RL success



Actor-Learner Architecture

Separate actors and learners to maximize throughput, leveraging CPU farms for simulation and GPUs for training



Low-Latency Serving

Deployment requires optimized inference, strict latency controls, and robust safety mechanisms

Questions? Resources: [Ray Project](#) | [CleanRL](#) | Spinning Up in RL